



ESTRUCT. DE DATOS Y ALGORITMOS
“PRIMER PARCIAL”

Estudiante:

Carlos M Delgado Rodriguez.

Profesor:

Fabian Chinchilla Mayorga

Año

2026

Proyecto 1: Simulador de Operaciones Numéricas con Undo/Redo (Implementación con ArrayList)

Descripción General

Desarrollar una aplicación en Java que simule un “acumulador” numérico interactivo. El usuario podrá aplicar operaciones de suma y resta (por ejemplo, +5, -3) y gestionar un historial dinámico para deshacer (**Undo**) y rehacer (**Redo**) estas operaciones.

El estudiante deberá simular el comportamiento de una pila (**LIFO: Last-In, First-Out**) utilizando la clase ArrayList, controlando conceptualmente las inserciones y extracciones siempre al final de la lista.

Objetivos Específicos

- Implementar dos ArrayList<Operacion> para gestionar el historial de **undo** y **redo**, simulando el comportamiento de una pila mediante la manipulación exclusiva del último elemento (`size() - 1`).
- Diseñar clases en Java que respeten la separación de responsabilidades, dividiendo el sistema en lógica de negocio (modelo) e interfaz de usuario (controlador/vista).
- Manejar entradas inválidas y mitigar errores de desbordamiento lógico o accesos a índices vacíos (*underflow*).
- Crear un menú interactivo continuo utilizando JOptionPane para todas las interacciones del usuario.

Requisitos y Alcance

- Java 11 o superior, proyecto estructurado con Maven o Gradle.
- Paquetes recomendados:
 - clase4.modelo (clases Acumulador y Operacion)
 - clase4.ui (clase SimuladorGUI encargada de los diálogos con JOptionPane)
- Persistencia exclusivamente en memoria dinámica (volátil).
- Operaciones soportadas: +N, -N, undo, redo, mostrar acumulador, mostrar historial de listas, salir.

Funcionalidades Mínimas y Lógica LIFO

1. **Operar:** introducir cadenas con el formato +N o -N (ej. +10, -4). Parsear el signo y el valor numérico, aplicar el cambio al acumulador, registrar la acción al final del ArrayList de *undo* (`.add(operacion)`) y *vaciar por completo* el ArrayList de *redo*.
2. **Undo (Deshacer):** revertir la última acción. Extraer y eliminar el último elemento en la

posición `size() - 1` de la lista de *undo*, aplicar su inversa matemática al acumulador y añadir dicha operación al final del `ArrayList` de *redo*.

3. **Redo (Rehacer):** volver a aplicar la última operación deshecha. Extraer y eliminar el último elemento en la posición `size() - 1` de la lista de *redo*, aplicarlo al acumulador y trasladarlo de vuelta al final del `ArrayList` de *undo*.
4. **Mostrar acumulador:** permitir visualizar el valor neto actual.
5. **Mostrar historial:** listar el contenido actual de ambos `ArrayList`, indicando cuál es el elemento que representa el tope y cuál es el fondo.
6. **Salir:** finalizar el ciclo de ejecución de forma limpia, incluyendo la acción de presionar el botón "Cancelar" en los cuadros de diálogo.

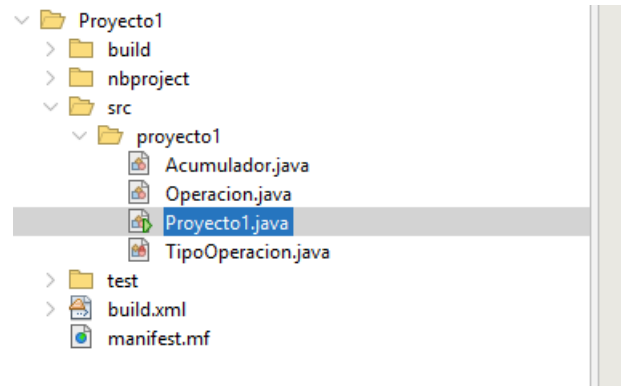
Importante: Tras realizar un *undo* seguido de una nueva operación manual, el historial del `ArrayList` de *redo* debe quedar completamente vacío (`.clear()`).

Entregables

- **Código Fuente Java:**
 - Estructura de paquetes organizada (modelo y ui).
 - Código limpio y libre de advertencias de compilación.
- **Capturas de pantalla** (mínimo 3 debidamente explicadas):
 - Ingreso de operación y actualización del acumulador.
 - Uso exitoso del mecanismo de Undo.
 - Visualización del historial de los `ArrayList` o uso del Redo.
- **Documento de Evidencias** (.docx o .pdf):
 - Encabezado institucional y título.
 - Descripción del diseño lógico implementado para simular LIFO con listas.
 - Las capturas de pantalla solicitadas.
 - ***Video corto (unos 5 minutos) mostrando el programa funcionando.***

Evidencia del Proyecto;

Con la base dada para realizar el proyecto se crea una cuarta clase Tipo de operación, esto para ser llamada Operación y definir el tipo de operación a realizar.



Operación:

En esta clase Operación es donde se realiza la operación sobre el acumulador, va ha almacenar el valor de la operación ya sea suma o resta y es donde permite tener la inversa utilizadas para rehacer o deshacer.

```
public class Operacion {  
  
    private int valor;  
    private TipoOperacion tipo;  
  
    public Operacion(int valor, TipoOperacion tipo) {  
        this.valor = valor;  
        this.tipo = tipo;  
    }  
  
    public int getValor() {  
        return valor;  
    }  
  
    public TipoOperacion getTipo() {  
        return tipo;  
    }  
  
    public Operacion inversa() {  
  
        if (tipo == TipoOperacion.SUMA) {  
            return new Operacion(valor, TipoOperacion.RESTA);  
        } else {  
            return new Operacion(valor, TipoOperacion.SUMA);  
        }  
    }  
  
    @Override  
    public String toString() {  
        if (tipo == TipoOperacion.SUMA) {  
            return "+" + valor;  
        } else {  
            return "-" + valor;  
        }  
    }  
}
```

Acumulador:

El Acumulador va a administrar el valor acumulado de las operaciones realizadas por el usuario y permite realizar operaciones de suma y resta, evidencia el historial de cambios mediante dos pilas (Undo y Redo), deshacer y rehacer operaciones, y mostrar el estado actual del acumulador junto con el contenido de ambas pilas.

```
public class Acumulador {  
  
    private int total;  
    private ArrayList<Operacion> undoStack;  
    private ArrayList<Operacion> redoStack;  
  
    public Acumulador() {  
        total = 0;  
        undoStack = new ArrayList<>();  
        redoStack = new ArrayList<>();  
    }  
  
    public int getTotal() {  
        return total;  
    }  
  
    public ArrayList<Operacion> getUndoStack() {  
        return undoStack;  
    }  
  
    public ArrayList<Operacion> getRedoStack() {  
        return redoStack;  
    }  
  
    public void aplicar(Operacion op) {  
  
        if (op.getTipo() == TipoOperacion.SUMA) {  
            total += op.getValor();  
        } else {  
            total -= op.getValor();  
        }  
  
        undoStack.add(op);  
  
        // Si se hace una nueva operación, se limpia el redo  
        redoStack.clear();  
  
        System.out.println("Resultado después de aplicar:");  
        System.out.println("Total: " + total);  
        System.out.println("Undo: " + undoStack);  
        System.out.println("Redo: " + redoStack);  
    }  
}
```

```
public void undo() {  
  
    if (undoStack.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "No hay operaciones para deshacer.");  
        return;  
    }  
  
    // Sacar la última operación  
    Operacion op = undoStack.remove(undoStack.size() - 1);  
  
    // Aplicar su inversa  
    Operacion inversa = op.inversa();  
  
    if (inversa.getTipo() == TipoOperacion.SUMA) {  
        total += inversa.getValor();  
    } else {  
        total -= inversa.getValor();  
    }  
  
    // Guardar la operación original en Redo  
    redoStack.add(op);  
  
    System.out.println("Resultado después de Undo:");  
    System.out.println("Total: " + total);  
    System.out.println("Undo: " + undoStack);  
    System.out.println("Redo: " + redoStack);  
}  
  
public void redo() {  
  
    if (redoStack.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "No hay operaciones para rehacer.");  
        return;  
    }  
  
    // Sacar la última operación del redo  
    Operacion op = redoStack.remove(redoStack.size() - 1);  
  
    // Aplicarla nuevamente  
    if (op.getTipo() == TipoOperacion.SUMA) {  
        total += op.getValor();  
    } else {  
        total -= op.getValor();  
    }  
  
    public void estado() {  
  
        String undo = "UNDO:\n";  
  
        if (undoStack.isEmpty()) {  
            undo += "Vacio\n";  
        } else {  
            undo += "Fondo\n";  
            for (Operacion op : undoStack) {  
                undo += op + "\n";  
            }  
            undo += "Tope\n";  
        }  
  
        String redo = "REDO:\n";  
  
        if (redoStack.isEmpty()) {  
            redo += "Vacio\n";  
        } else {  
            redo += "Fondo\n";  
            for (Operacion op : redoStack) {  
                redo += op + "\n";  
            }  
            redo += "Tope\n";  
        }  
  
        JOptionPane.showMessageDialog(  
            null,  
            "TOTAL ACUMULADO: " + total + "\n\n"  
            + undo + "\n"  
            + redo  
        );  
    }  
}
```

Proyecto

La clase Proyecto1 es la clase principal de la aplicación. Se encarga de mostrar el menú al usuario, recibir las opciones seleccionadas y coordinar las acciones del programa mediante la clase Acumulador, permitiendo agregar operaciones, deshacer, rehacer, consultar el acumulador y visualizar el historial.

Acá vamos a tener un menú realizado por un switch y es donde va a llamar las clases operación y acumulador y mostrara los resultados.

```
public class Proyecto1 {

    private static Acumulador acu = new Acumulador();

    /**
     * @param args the command line arguments
     */
    public static String mostrarMenu() {
        String opciones[] = {
            "Salir",
            "Mostrar el acumulador",
            "Agregar",
            "Undo-DESHACER",
            "Redo-REHACER",
            "Estado-HISTORIAL"
        };
        String menu = "";
        int i = 0;
        for (String o : opciones) {
            menu += (i++) + ". " + o + "\n";
        }
        return menu;
    }

    public static void main(String[] args) {
        while (true) {
            String input = JOptionPane.showInputDialog(null, mostrarMenu());
            if (input == null) {
                return;
            }
            try {
                int option = Integer.parseInt(input);

                switch (option) {
                    case 0:
                        return;
                    case 1:
                        int acumulador = acu.getTotal();
                        JOptionPane.showMessageDialog(null,
                            "El valor del acumulador es: " + acumulador);
                        break;
                    case 2:
                        manejarOperador();
                        break;
                    case 3:
                        manejarUndo();
                        break;
                }
            }
        }
    }
}
```

```

        case 4:
            manejarRedo();
            break;
        case 5:
            acu.estado();
            break;

        default:
            JOptionPane.showMessageDialog(null,
                "Opcion no encontrada",
                "Error", JOptionPane.WARNING_MESSAGE);
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(null, "Debe ingresar un valor numerico");
}
}

private static void manejarUndo() {
    acu.undo();
}

private static void manejarRedo() {
    acu.redo();
}

private static void manejarOperador() {
    String input = JOptionPane.showInputDialog(
        "Ingrese una operación.\nEjemplos:\n+10\n-5"
    );
    if (input == null || input.isEmpty()) {
        return;
    }

    try {
        int valor = Integer.parseInt(input);

        TipoOperacion tipo;

        if (valor >= 0) {
            tipo = TipoOperacion.SUMA;
        } else {
            tipo = TipoOperacion.RESTA;
        }

        Operacion op = new Operacion(Math.abs(valor), tipo);
        acu.aplicar(op);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(null, "Debe ingresar un número válido");
    }
}

```